Deep Learning

Lecture 2 — Neurons Before Networks: Perceptron, Linear Models, Gradient Descent

Przemysław Witaszczyk

Jagiellonian University
Faculty of Physics, Astronomy and Applied Computer Science

Where we are

- Lecture 1 established the methodological rules of the course: the dataset contract, metrics fixed before training, cross-validation, the leakage-proof pipeline, and the bias–variance ledger. Every model there was a black box.
- The simplest of those boxes are opened here. Everything still runs under Lecture 1's rules — every experiment below splits its data, states its metric, and shows its frozen output.
- Machinery the entire course reuses receives its definitive treatment here: the training-loop skeleton, gradient descent, the maximum-likelihood origin of loss functions, the softmax, and the ridge/lasso penalties. Later lectures quote these results rather than re-derive them.

The plan for today: the mechanical core of deep learning

- The **perceptron** is treated first, as the course's earliest learning algorithm; we state its update rule, examine its convergence, and prove — rather than assert — its hard built-in limitation.
- **Adaline** then makes the decisive change: replacing a step by a continuous output turns classification into differentiable *optimization*. Everything after this change runs on gradients.
- **Gradient descent** is derived from the directional derivative, and its failure modes are computed by hand.
- Loss functions emerge from **maximum likelihood**: we show why squared error, cross-entropy, and the softmax are theorems rather than choices.
- The final act treats **linear, ridge, lasso, logistic, and softmax regression** — the last of which is a neural network with zero hidden layers.

Symbols added today

- A linear unit has weights $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$; its **score** (pre-activation) is $z = \mathbf{w}^\top \mathbf{x} + b$.
- $\sigma(\cdot)$ denotes the logistic sigmoid, defined in this lecture; η is the learning rate announced in Lecture 1's symbol card.
- Label conventions differ between acts, and we will flag the switch in words: the perceptron and Adaline use $y \in \{-1, +1\}$ (the classical convention, which makes their update rules clean); the probabilistic models of the final act use $y \in \{0, 1\}$.
- For regression in matrix form, $\mathbf{X}$ is the $m \times (d+1)$ **design matrix** whose first column is all ones, and $\boldsymbol{\theta}$ stacks the bias and the weights, so that scores are $\mathbf{X}\boldsymbol{\theta}$.

The setting, and two dated facts

- The task is binary classification with a *linear score*: the model computes $z = \mathbf{w}^\top \mathbf{x} + b$ and predicts by its sign. Learning means choosing $\mathbf{w}$ and b from labeled examples.
- Two dated facts locate the idea historically. In 1943 McCulloch and Pitts showed that thresholded weighted sums can implement logical functions. In 1957 Rosenblatt supplied what was missing — an *algorithm that learns the weights from data* — and named it the perceptron.
- We treat the perceptron purely as that: an algorithm with a convergence theorem and a precise limitation. Both books open this topic with biological narratives; this course does not use them.

The model: a thresholded linear score

With labels $y \in \{-1, +1\}$, the perceptron predicts

$$\hat{y} = \begin{cases} +1 & \text{if } z = \mathbf{w}^\top \mathbf{x} + b \geq 0, \\ -1 & \text{otherwise.} \end{cases}$$

- The set $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ is a hyperplane — a line in two dimensions — called the **decision boundary**.
- The weight vector $\mathbf{w}$ is *normal* to the boundary: for any two boundary points $\mathbf{x}, \mathbf{x}'$ we have $\mathbf{w}^\top (\mathbf{x} - \mathbf{x}') = 0$. The bias b shifts the boundary away from the origin.
- The classifier is therefore a geometric object: it can only ever separate the plane into two half-planes. This observation is strengthened into a proved limitation at the end of this act.

The learning rule

The perceptron visits the training examples one at a time and updates only when it errs:

Perceptron update (learning rate η)

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}, \qquad b \leftarrow b + \eta\,(y - \hat{y}).$$

- When the prediction is correct, $y - \hat{y} = 0$ and nothing moves. When it is wrong, $y - \hat{y} = \pm 2$, and the update adds $\pm 2\eta\,\mathbf{x}$ to the weights.
- The direction is the informative part: after a mistake on a positive example, the score $\mathbf{w}^\top \mathbf{x} + b$ *increases* on that example (by $2\eta(\mathbf{x}^\top \mathbf{x} + 1) > 0$) — the boundary moves toward classifying it correctly.
- This is *error-driven* learning, defined without a loss function and without a gradient; the next act supplies both.

One update, computed by hand

We take $\mathbf{w} = (0.5, -1)$, $b = 0$, $\eta = 0.1$, and the positive example $\mathbf{x} = (1, 2)$, $y = +1$. All numbers verified in `code/verify_numerics.py`.

$$z = \mathbf{w}^\top \mathbf{x} + b = 0.5 \cdot 1 + (-1) \cdot 2 + 0 = -1.5,$$

so $\hat{y} = -1$ — an error — and the update magnitude is $\eta\,(y - \hat{y}) = 0.1 \cdot (1 - (-1)) = 0.2$. Therefore

$$\mathbf{w} \leftarrow (0.5, -1) + 0.2 \cdot (1, 2) = (0.7, -0.6),$$
$$b \leftarrow 0 + 0.2 = 0.2.$$

The new score on the same example is $0.7 \cdot 1 - 0.6 \cdot 2 + 0.2 = -0.3$: still negative, but moved from -1.5 toward the boundary. Repeated visits complete the correction.

Convergence, and the non-separable case

- The **perceptron convergence theorem** — quoted here, with a proof sketch in the headroom section — states that if the training data is *linearly separable*, the perceptron makes finitely many mistakes: at most $(R/\gamma)^2$, where R bounds the example norms and γ is the separation margin.
- The bound has readable structure: hard problems are those with *small margins* relative to the data's scale.
- In the non-separable case, by contrast, the algorithm never settles — some example is always misclassified, so updates continue forever. There is no notion of "best imperfect boundary" because there is no loss being minimized. This deficiency motivates the entire next act.

The perceptron in code, I: the class

The implementation is a direct transcription of the two formulas (full script: `code/01_perceptron.py`).

```
class Perceptron:
    def __init__(self, eta=0.1, n_epochs=10):
        self.eta, self.n_epochs = eta, n_epochs

    def score(self, X):
        return X @ self.w + self.b

    def predict(self, X):
        return np.where(self.score(X) >= 0.0, 1, -1)
```

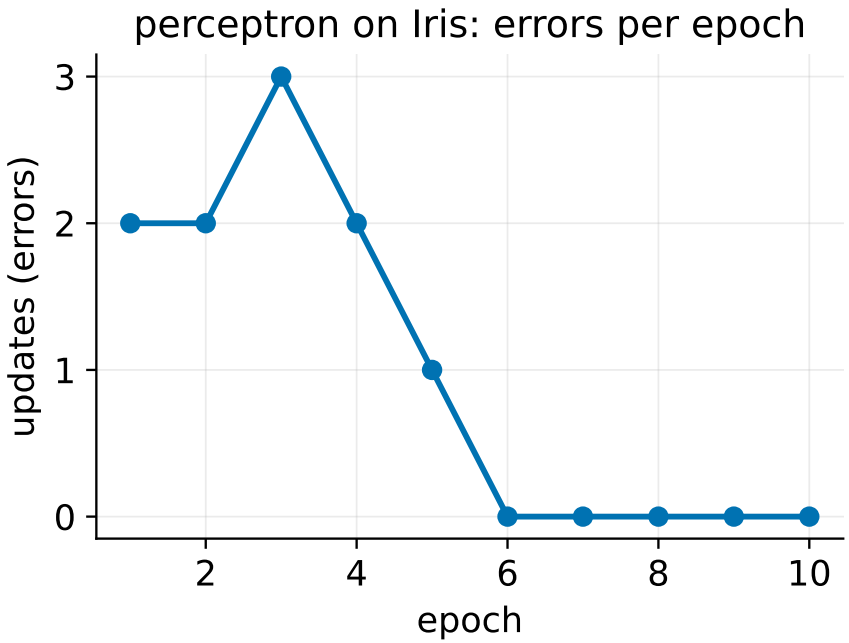
- The score and the thresholded prediction are kept separate — the same separation the mathematics makes, and the point at which Adaline changes the algorithm.

The perceptron in code, II: the training loop

```
def fit(self, X, y):
    self.w = np.zeros(X.shape[1]); self.b = 0.0
    self.errors_ = []
    for _ in range(self.n_epochs):
        errors = 0
        for xi, yi in zip(X, y):
            update = self.eta * (yi - self.predict(xi))
            self.w += update * xi   # zero when correct
            self.b += update
            errors += int(update != 0.0)
        self.errors_.append(errors)
    return self
```

- One **epoch** is one pass through the training set. The per-epoch error count is recorded — it is the convergence diagnostic of the frozen run that follows.

The frozen run: convergence on Iris

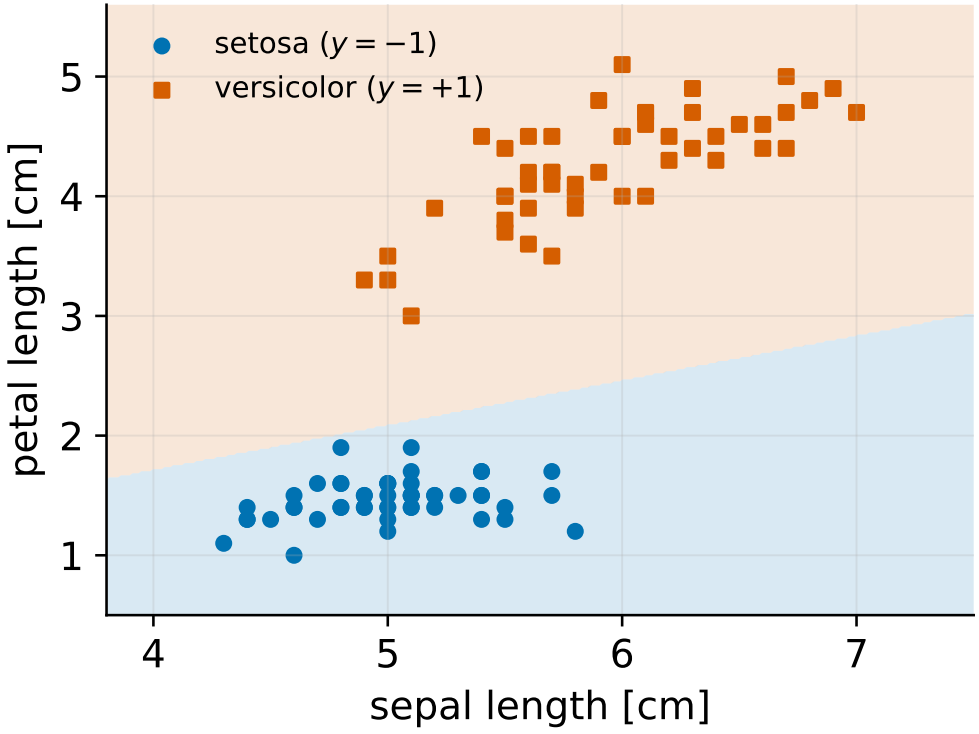


The data is two Iris species with two features — a linearly separable problem, so the theorem applies:

```
errors per epoch:
[2, 2, 3, 2, 1, 0, 0, 0, 0, 0]
final w = [-0.68  1.82]
b = -0.40
```

After five epochs the boundary makes no further mistakes, and the weights never move again — exactly the finite-mistake behavior the theorem guarantees.

The learned boundary



Decision regions of the converged perceptron on the two separable Iris classes.

Generated by `code/01_perceptron.py`.

- The boundary is a line, as the geometry requires, and every training example lies on its correct side.
- Convergence stops at the *first* separating line the updates happen to reach — not at any "best" one. The perceptron has no preference among separators, because it has no loss.

The limitation: XOR is not linearly separable

Consider four points with the "exclusive or" labeling:

x_1	x_2	y
0	0	$-$
1	1	$-$
1	0	$+$
0	1	$+$

Claim

No weights w_1, w_2, b make $\mathbf{1}[w_1 x_1 + w_2 x_2 + b \geq 0]$ reproduce this labeling.

The proof is four lines of arithmetic, and we give it in full.

The proof, in full

Suppose such w_1, w_2, b existed. The two positive points require

$$w_1 + b \geq 0 \qquad \text{and} \qquad w_2 + b \geq 0,$$

and adding these gives $w_1 + w_2 + 2b \geq 0$. The two negative points require

$$b < 0 \qquad \text{and} \qquad w_1 + w_2 + b < 0,$$

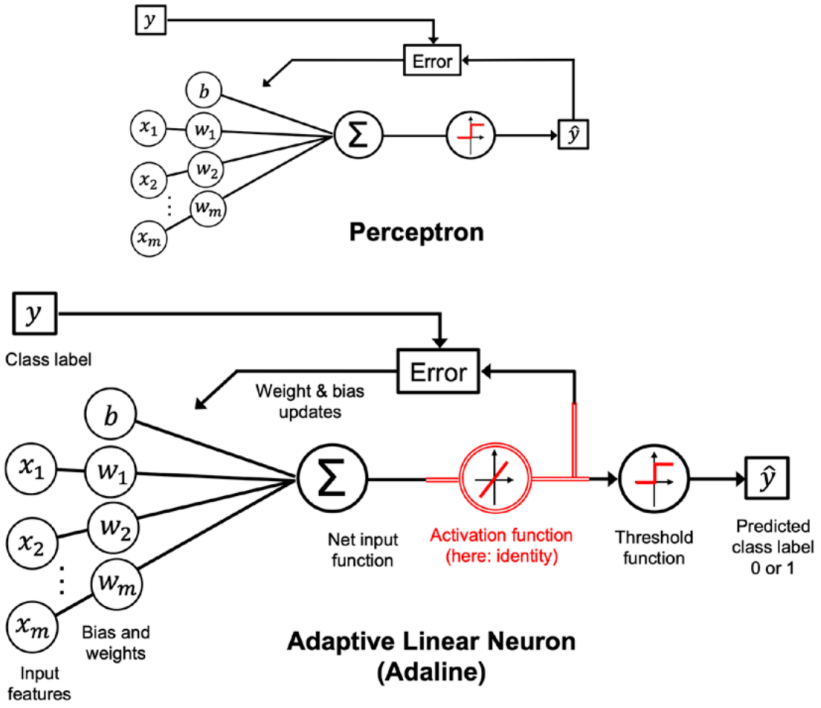
and adding *these* gives $w_1 + w_2 + 2b < 0$. The same quantity cannot be both nonnegative and negative, so no such weights exist. □

- The failure is not about the algorithm: *no* linear threshold unit, however trained, computes XOR. The model family is too small — Lecture 1's bias, in its purest form.
- This limitation is resolved in Lecture 3 by a network of *three* such units, and the solution is exhibited there explicitly.

Why the perceptron cannot be improved gradually

- The perceptron's prediction is a step function of the score, and therefore piecewise constant in the parameters: an infinitesimal change of $\mathbf{w}$ almost never changes any prediction, and occasionally flips one discontinuously.
- A quantity that is flat almost everywhere provides no *direction of improvement* — its derivative is zero wherever it exists. The error count registers that the parameters are wrong, but not in which direction to move.
- The repair, due to Widrow and Hoff (1960, the "Adaline" — adaptive linear neuron), is to drop the threshold during *learning* and treat the continuous score z itself as the output. The threshold returns only at *prediction* time.

The change, in one picture



Raschka et al. (2022), Fig. 2.9. The book writes labels as 0/1; our convention in this act is ± 1.

- The perceptron (top) compares the *thresholded* prediction with the label. Adaline (bottom) compares the *continuous* score with the label, and only that difference drives learning.
- This is the design change on which everything after it depends: with a continuous output, a differentiable loss exists, and classification becomes optimization.

Adaline's objective, and its gradient in full

Adaline minimizes the mean squared error (MSE) between score and label,

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(y^{(i)} - z^{(i)}\right)^2, \qquad z^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} + b.$$

Differentiating with respect to one weight w_j, by the chain rule:

$$\begin{aligned} \frac{\partial J}{\partial w_j} &= \frac{1}{2m} \sum_i 2\left(y^{(i)} - z^{(i)}\right) \cdot \frac{\partial}{\partial w_j}\left(y^{(i)} - z^{(i)}\right) \\ &= -\frac{1}{m} \sum_i \left(y^{(i)} - z^{(i)}\right) x_j^{(i)}, \end{aligned}$$

since $\partial z^{(i)} / \partial w_j = x_j^{(i)}$. The same steps with $\partial z / \partial b = 1$ give

$$\frac{\partial J}{\partial b} = -\frac{1}{m} \sum_i \left(y^{(i)} - z^{(i)}\right).$$

Every term is an *error times the input that caused it* — a pattern that returns, layer by layer, in backpropagation.

The training-loop skeleton, named

Updating against the gradient closes the loop, and the loop receives a permanent name — every model in this course, to Lecture 13, runs it:

The training-loop skeleton

forward (compute z, $\hat{y}$) $\rightarrow$ **loss** (compute J) $\rightarrow$ **backward** (compute ∇J) $\rightarrow$ **step** ($\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla J$)

- For Adaline, each stage is one line of algebra. For a transformer, each stage is a large computation. The skeleton is identical.
- Lecture 4 will show that PyTorch is precisely this skeleton with the backward stage automated.
- The step direction $-\nabla J$ requires its own justification, which is given in the next act.

Why the negative gradient: the directional derivative

For a differentiable $J : \mathbb{R}^p \to \mathbb{R}$ and a unit vector $\vec{u}$, the **directional derivative** measures the instantaneous change of J when moving along $\vec{u}$:

$$D_{\vec{u}} J(\boldsymbol{\theta}) = \lim_{h \to 0} \frac{J(\boldsymbol{\theta} + h\vec{u}) - J(\boldsymbol{\theta})}{h} = \nabla J(\boldsymbol{\theta})^\top \vec{u},$$

the last equality by the chain rule applied to $h \mapsto J(\boldsymbol{\theta} + h\vec{u})$. By the Cauchy–Schwarz inequality,

$$\left| \nabla J^\top \vec{u} \right| \;\le\; \|\nabla J\| \, \|\vec{u}\| \;=\; \|\nabla J\|,$$

with equality exactly when $\vec{u}$ is parallel to ∇J.

- The gradient direction is therefore the direction of *steepest ascent*, and $-\nabla J$ the direction of steepest descent. This is a theorem, not a convention.

The algorithm, and its single hyperparameter

Gradient descent (GD)

Initialize $\boldsymbol{\theta}$; repeat until convergence: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla J(\boldsymbol{\theta})$.

- The **learning rate** η scales the step. It is the hyperparameter to which training is most sensitive, and it is searched on a log scale (Lecture 1's rule).
- Locally, each step decreases J whenever η is small enough: to first order, $J(\boldsymbol{\theta} - \eta \nabla J) \approx J(\boldsymbol{\theta}) - \eta \|\nabla J\|^2$.
- What "small enough" means, and what happens otherwise, is established on the simplest possible landscape, worked through below with every number computed by hand.

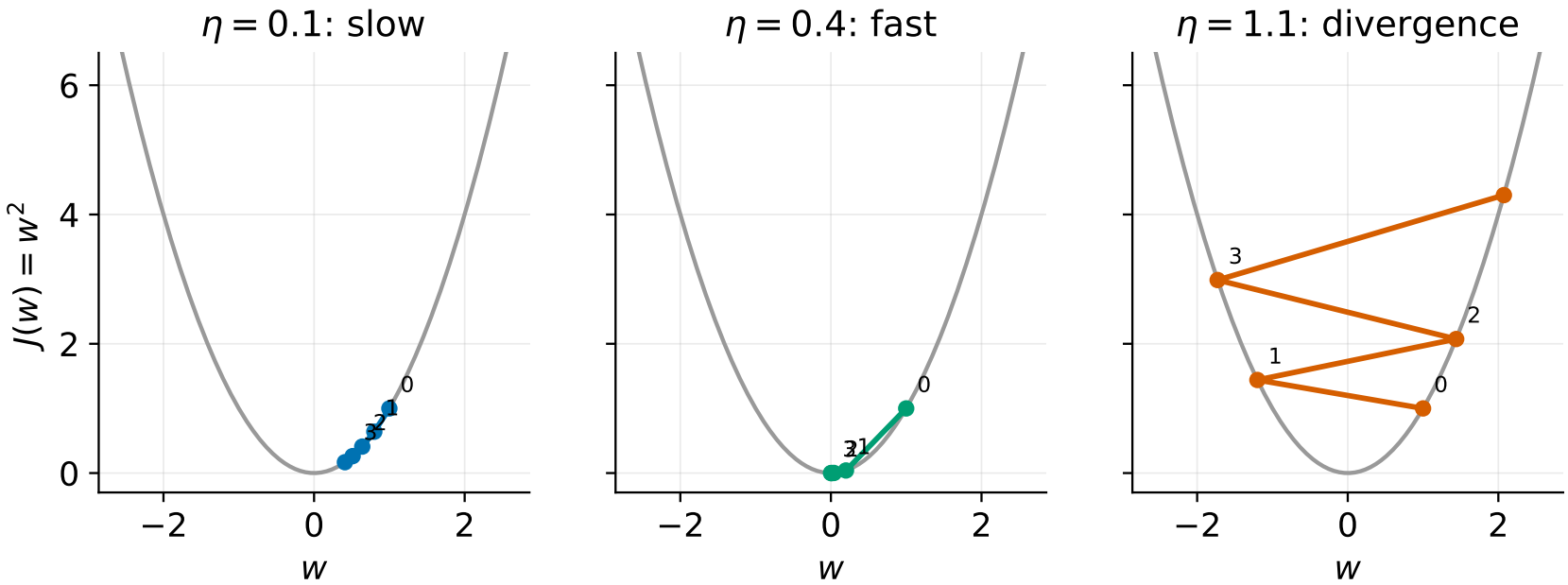
Learning-rate pathologies, computed by hand

Take $J(w) = w^2$, so $\nabla J = 2w$ and one step is $w \leftarrow w - 2\eta w = (1 - 2\eta)\, w$: each step multiplies w by the constant $(1 - 2\eta)$. Starting from $w_0 = 1$ (verified in `code/verify_numerics.py`):

	w_0	w_1	w_2	w_3	w_4
$\eta = 0.1$: factor 0.8	1	0.8	0.64	0.512	0.4096
$\eta = 1.1$: factor -1.2	1	-1.2	1.44	-1.728	2.0736

- Convergence requires $|1 - 2\eta| < 1$, that is $\eta < 1$ for this problem: too small crawls, too large overshoots the minimum with growing, sign-flipping amplitude.
- The threshold depends on the *curvature* of J — a fact that becomes central for ill-conditioned problems in Lecture 6.

The three regimes, drawn



The hand-computed iterates of the two pathological learning rates (plus a well-chosen $\eta = 0.4$), overlaid on $J(w) = w^2$. Numbered points are successive steps. Generated by `code/03_gd_quadratic.py`.

- The middle panel shows what a good learning rate yields: with $\eta = 0.4$ the factor is 0.2, and four steps reach 0.0016.
- These three regimes — crawl, converge, diverge — reappear on every loss surface of the course.

One gradient step on a real MSE, by hand

A regression with one parameter: $\hat{y} = wx$ on the two examples $(1, 2)$ and $(2, 3)$, with $J(w) = \frac{1}{2m}\sum_i (wx^{(i)} - y^{(i)})^2$ and $\eta = 0.1$. Starting at $w = 0$ (verified in `code/verify_numerics.py`):

$$\nabla J(0) = \tfrac{1}{2}\big[(0-2)\cdot 1 + (0-3)\cdot 2\big] = \tfrac{1}{2}(-2-6) = -4,$$
$$w \leftarrow 0 - 0.1\cdot(-4) = 0.4,$$
$$\nabla J(0.4) = \tfrac{1}{2}\big[(0.4-2)\cdot 1 + (0.8-3)\cdot 2\big] = \tfrac{1}{2}(-1.6-4.4) = -3,$$
$$w \leftarrow 0.4 - 0.1\cdot(-3) = 0.7.$$

- The gradient shrinks as the fit improves (-4, then -3), so the steps shorten by themselves — gradient descent decelerates into minima even at fixed η.

Batch, stochastic, and mini-batch

The gradient of the mean loss is a mean of per-example gradients, and this licenses three variants:

- **Batch** gradient descent uses all m examples per step: the exact gradient, at cost proportional to m per step.
- **Stochastic** gradient descent (SGD) uses a single example per step: an unbiased but noisy estimate. The path jitters, which costs precision near the minimum but yields m inexpensive steps per epoch — and the noise can carry the iterate out of poor regions.
- **Mini-batch** gradient descent uses a few dozen examples per step: most of the noise reduction at a fraction of the cost, and the format modern hardware parallelizes best. From Lecture 4 onward, this is the course default.

One **epoch** is one full pass through the training data in any of the three schemes.

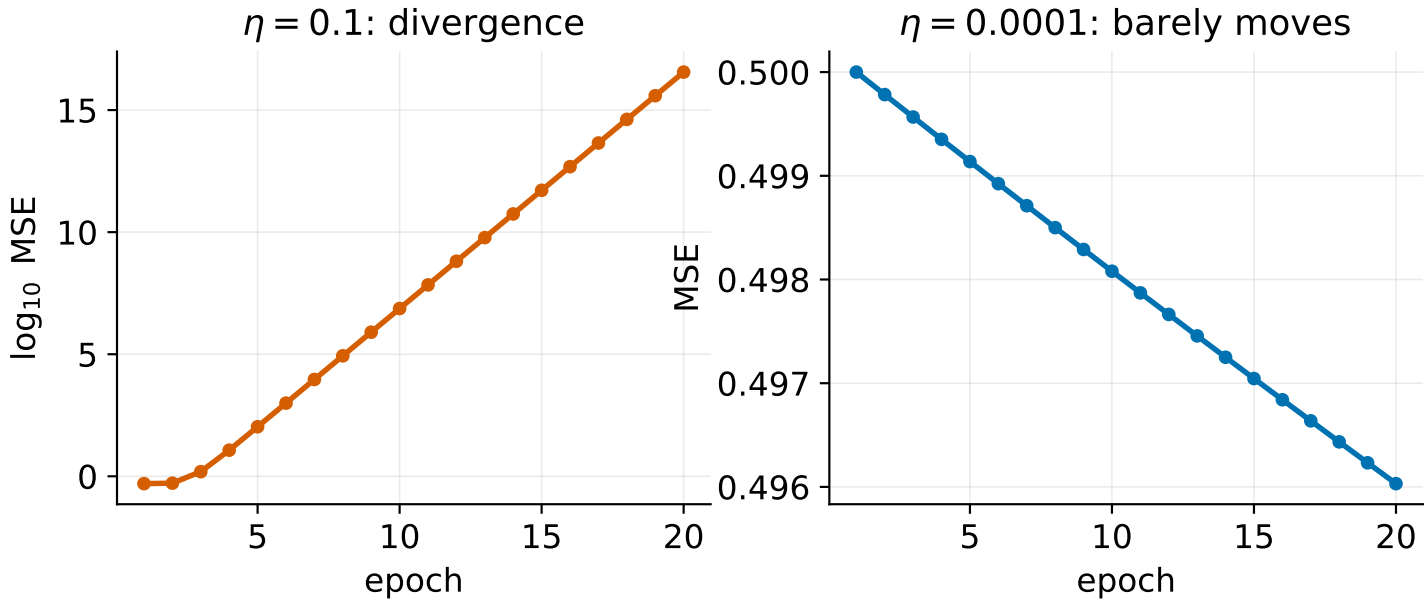
Adaline in code

The skeleton, transcribed — forward, loss, backward, and step are visible as lines (full script: `code/02_adaline.py`):

```
class AdalineGD:
    def fit(self, X, y):
        self.w = np.zeros(X.shape[1]); self.b = 0.0
        for _ in range(self.n_epochs):
            z = X @ self.w + self.b              # forward
            err = y - z
            loss = (err ** 2).mean() / 2         # loss
            grad_w = -X.T @ err / len(y)         # backward
            self.w -= self.eta * grad_w          # step
            self.b += self.eta * err.mean()
        return self
```

- Full-batch updates: one gradient step per epoch, using every example — the "batch" end of the three-variant spectrum.

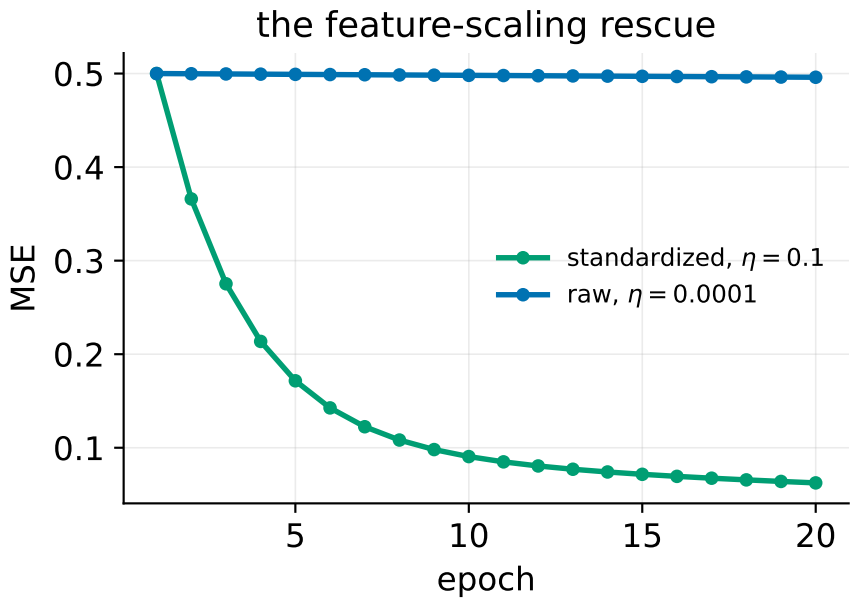
The frozen run: both pathologies at once




```
eta=0.1    (raw): epoch-1 loss 0.50,   epoch-20 loss 3.6e+16
eta=0.0001 (raw): epoch-1 loss 0.5000, epoch-20 loss 0.4960
```

- On the raw Iris features, $\eta = 0.1$ diverges by sixteen orders of magnitude while $\eta = 0.0001$ improves the loss by less than one percent in twenty epochs.
- The window of workable learning rates is, apparently, narrow. The scaling experiment that follows shows that the data — not the algorithm — is responsible.

The feature-scaling correction



Standardizing the two features (mean 0, variance 1 — learned on the training data, per the contract) removes both pathologies:

```
eta=0.1 (standardized):
loss epoch 1  = 0.5000
loss epoch 20 = 0.0624
```

The same $\eta = 0.1$ that diverged on raw features now converges smoothly.

- This confirms Lecture 1's statement that gradient-trained models *require* scaling: features of unequal scale curve the loss surface unequally, and no single η suits both directions.
- The full geometry of this statement (conditioning, curvature) is developed in Lectures 5 and 6; today the experiment suffices.

Where losses come from

Adaline's squared error was inherited without justification. This act removes the arbitrariness: the standard losses of machine learning are *consequences* of one principle.

Maximum likelihood estimation (MLE)

Model the data as draws from a distribution $p(y \mid \mathbf{x}; \boldsymbol{\theta})$. Choose the parameters under which the observed data is most probable:

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\big) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ln p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\big).$$

- The product is licensed by Lecture 1's i.i.d. (independent, identically distributed) assumption, and the logarithm turns it into a sum without moving the argmax (the logarithm is strictly increasing).
- Minimizing the *negative* log-likelihood is then a loss minimization — and each choice of noise model yields a loss.

Gaussian noise yields squared error

Model the target as the prediction plus Gaussian noise: $y = f_{\boldsymbol{\theta}}(\mathbf{x}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, so that

$$p(y \mid \mathbf{x}; \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(- \frac{(y - f_{\boldsymbol{\theta}}(\mathbf{x}))^2}{2\sigma^2} \Big).$$

Taking the negative log-likelihood of the dataset:

$$-\sum_i \ln p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\big) = \frac{m}{2} \ln(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_i \big(y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})\big)^2.$$

The first term does not depend on $\boldsymbol{\theta}$, and the positive factor $1/2\sigma^2$ does not move the argmin. Therefore

$$\hat{\boldsymbol{\theta}}_{\mathrm{MLE}} = \arg\min_{\boldsymbol{\theta}} \sum_i \big(y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})\big)^2.$$

Minimizing squared error is maximum likelihood under Gaussian noise. Adaline's loss encoded an unstated distributional assumption.

Laplace noise yields absolute error

The same computation with heavier-tailed noise, $\varepsilon \sim \mathrm{Laplace}(0, s)$ with density $p(\varepsilon) = \frac{1}{2s} e^{-|\varepsilon|/s}$, gives

$$-\sum_i \ln p\left(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\right) = m \ln(2s) + \frac{1}{s} \sum_i \left| y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right|,$$

so the maximum-likelihood fit minimizes the sum of *absolute* errors.

- This completes Lecture 1's forward reference: the root-mean-squared and mean absolute errors (RMSE and MAE, Lecture 1's regression metrics) are not interchangeable conventions but different assumptions about the noise. Squared error assumes thin Gaussian tails and is therefore sensitive to outliers; absolute error assumes occasional large deviations and is far less affected by them.
- Choosing a loss *is* choosing a noise model. This reading guides every loss the course introduces from here on.

The sigmoid, and its derivative

Classification needs a probability in $(0,1)$ built from a score in $\mathbb{R}$. The **logistic sigmoid**

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

is strictly increasing, maps 0 to $\frac{1}{2}$, and satisfies $\sigma(-z) = 1 - \sigma(z)$. Its derivative has a closed form we will use repeatedly:

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\,\big(1-\sigma(z)\big),$$

where the last step uses $\frac{e^{-z}}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z)$.

- The derivative is largest at $z = 0$ (value $\frac{1}{4}$) and vanishes in both tails — a small fact with large consequences for deep networks, revisited as the vanishing-gradient problem in Lectures 3 and 5.

Bernoulli labels yield binary cross-entropy

With labels $y \in \{0, 1\}$ (the convention switch announced at the start), model $p(y{=}1 \,|\, \mathbf{x}) = \hat{p} = \sigma(z)$. The Bernoulli likelihood of one observation is

$$p(y \,|\, \mathbf{x}; \boldsymbol{\theta}) = \hat{p}^{\,y}(1 - \hat{p})^{1-y},$$

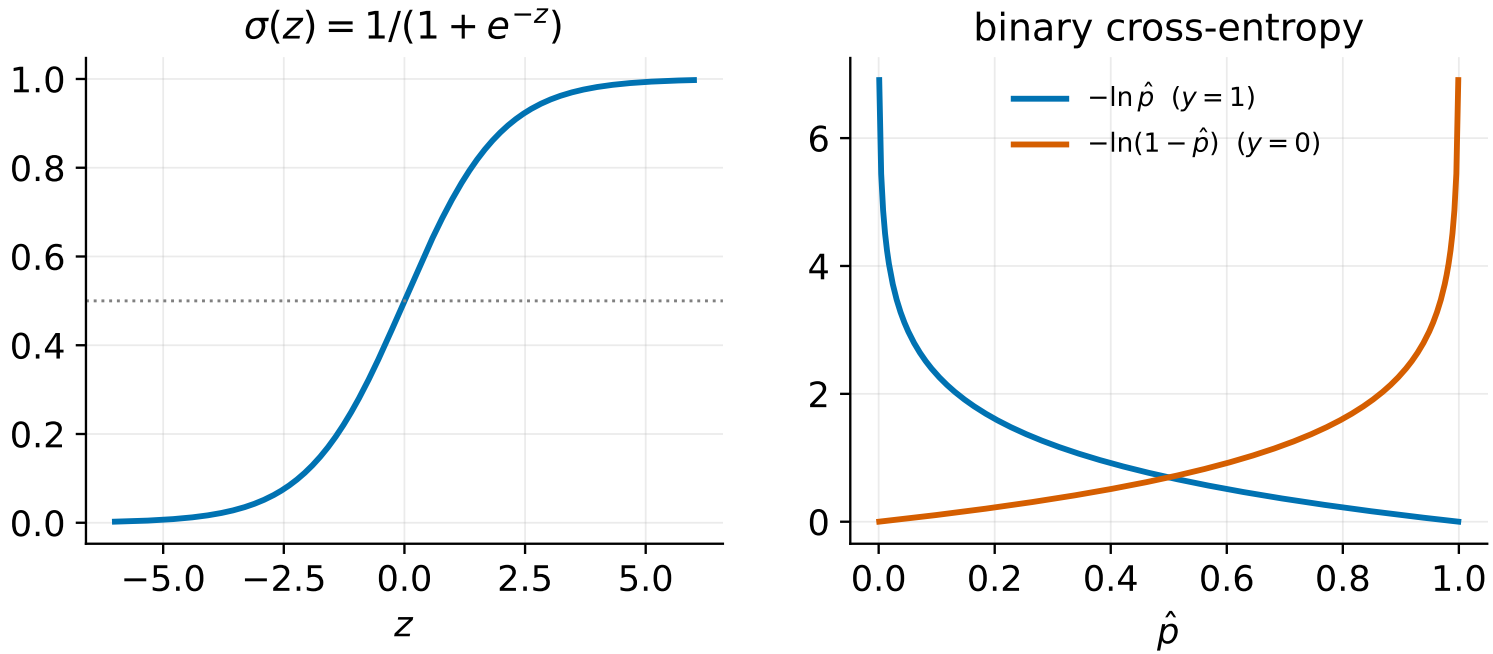
which reads: probability $\hat{p}$ if $y = 1$, probability $1 - \hat{p}$ if $y = 0$. The negative log-likelihood of the dataset is therefore

$$J(\boldsymbol{\theta}) = -\sum_{i=1}^{m} \left[y^{(i)} \ln \hat{p}^{(i)} + \left(1 - y^{(i)}\right) \ln \left(1 - \hat{p}^{(i)}\right) \right],$$

the **binary cross-entropy** (BCE).

- Each example activates exactly one of the two terms. A confident wrong prediction ($\hat{p} \to 0$ while $y = 1$) is penalized without bound — the loss diverges logarithmically.

The two branches of the loss, drawn



The sigmoid (left) and the two BCE branches (right). Generated by `code/04_losses_mle.py`.

- A spot value, verified in `code/verify_numerics.py`: for $z = 0.5$ we get $\sigma(0.5) = 0.6225$, and if the true label is $y = 1$ the loss is $-\ln 0.6225 = 0.474$.
- The loss is gentle near correct confident predictions and steep near wrong confident ones — exactly the asymmetry a training signal should have.

The softmax: probabilities for K classes

For K classes, the model produces one score per class, $\mathbf{z} = (z_1, \ldots, z_K)$, and the **softmax** converts scores to a probability vector. The softmax receives its definitive treatment here; it returns in every later architecture, up to attention weights in Lecture 9.

Definition

$$\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K.$$

- Each component is positive and the components sum to one, so the output is a genuine probability distribution for *any* real scores.
- Exponentiation makes the map order-preserving — the largest score gets the largest probability — and sharpens differences: score gaps become probability ratios, since $\mathrm{softmax}(\mathbf{z})_k / \mathrm{softmax}(\mathbf{z})_l = e^{z_k - z_l}$.

Two softmax properties that matter in practice

- **Shift invariance**: adding one constant c to every score changes nothing,

$$\frac{e^{z_k+c}}{\sum_j e^{z_j+c}} = \frac{e^c\, e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

  Only score *differences* carry meaning.

- **Numerical stability** follows from that same invariance: e^{1000} overflows floating point, so one subtracts the maximum score first — mathematically a no-op, numerically essential. The frozen demonstration (`code/04_losses_mle.py`):

```
logits: [1000. 1001. 1002.]
naive softmax:   [nan nan nan]  (exp overflowed to inf -> nan)
shifted softmax: [0.09   0.2447 0.6652]
```

Categorical labels yield softmax cross-entropy

Encode the true class c as an indicator vector $\mathbf{y}$ with $y_c = 1$ and zeros elsewhere. The categorical likelihood of one observation is

$$p(\mathbf{y} \mid \mathbf{x}; \boldsymbol{\theta}) = \prod_{k=1}^{K} \hat{p}_k^{\,y_k} = \hat{p}_c, \qquad \hat{p} = \operatorname{softmax}(\mathbf{z}),$$

and the negative log-likelihood of the dataset is the **(softmax) cross-entropy**

$$J(\boldsymbol{\theta}) = -\sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \ln \hat{p}_k^{(i)} = -\sum_{i=1}^{m} \ln \hat{p}_{c^{(i)}}^{(i)}.$$

- In words: the loss on one example reduces to the negative log of the probability assigned to the *true* class. All other classes enter only through the softmax's normalization.
- For $K = 2$ this reduces exactly to the BCE obtained from the Bernoulli likelihood — one derivation family, two class counts.

Softmax cross-entropy, computed by hand

Take three classes with scores $\mathbf{z} = (2, 0, 1)$ and true class 3. All numbers verified in `code/verify_numerics.py`.

$$e^2 = 7.389, \qquad e^0 = 1.000, \qquad e^1 = 2.718, \qquad \textstyle\sum_j e^{z_j} = 11.107,$$

$$\hat{\mathbf{p}} = \left(\tfrac{7.389}{11.107},\ \tfrac{1.000}{11.107},\ \tfrac{2.718}{11.107} \right) = (0.665,\ 0.090,\ 0.245),$$

$$J = -\ln \hat{p}_3 = -\ln 0.245 = 1.408.$$

- The model assigns the highest probability to class 1 while the true class is 3, and the loss quantifies that error. Had the true class been class 1, the loss would have been $-\ln 0.665 = 0.408$ — one unit smaller, for one unit of score difference.
- This tiny computation is the complete per-token loss of a large language model, with $K \approx 10^5$ classes — the objective that Lecture 10 trains at scale.

The loss table (the course's MLE principle)

output model	noise / label model	resulting loss
real number	Gaussian	squared error (MSE)
real number	Laplace	absolute error (MAE)
probability $\sigma(z)$	Bernoulli	binary cross-entropy
$\mathrm{softmax}(\mathbf{z})$	categorical	cross-entropy

- The pattern is general: *an output distribution is chosen, and the loss follows as its negative log-likelihood*. No entry in this table is a matter of convention.
- The same principle recurs throughout the course: the per-token language-model loss (Lectures 9–10), the variational bound of the VAE (Lecture 12), and the diffusion objective (Lecture 13) are all the same principle in richer models.

Linear regression in matrix form

Linear regression is the Gaussian-MLE problem for a linear model. Stack the data: let $\mathbf{X}$ be the $m \times (d+1)$ design matrix whose i-th row is $(1, \mathbf{x}^{(i)\top})$, and let $\boldsymbol{\theta}$ stack $(b, \mathbf{w})$, so all scores at once are $\mathbf{X}\boldsymbol{\theta}$. The objective is

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 = \frac{1}{2m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)}\right)^2,$$

writing $\mathbf{x}^{(i)}$ for the padded row. Differentiating with respect to component θ_j:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i} \left(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)}\right) x_j^{(i)},$$

and stacking the p components reassembles a matrix product:

$$\nabla J(\boldsymbol{\theta}) = \frac{1}{m} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}).$$

The normal equation

The objective is a convex quadratic, so its minimum is where the gradient vanishes:

$$\mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = 0 \quad \Longleftrightarrow \quad \mathbf{X}^\top\mathbf{X}\,\boldsymbol{\theta} = \mathbf{X}^\top\mathbf{y},$$

and when $\mathbf{X}^\top\mathbf{X}$ is invertible,

Normal equation

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^\top\mathbf{X}\right)^{-1}\mathbf{X}^\top\mathbf{y}.$$

- Linear regression is thus solvable *in closed form* — no iterations, no learning rate. This is exceptional; almost nothing after this lecture has a closed form.
- $\mathbf{X}^\top\mathbf{X}$ fails to be invertible when features are linearly dependent (or $d + 1 > m$); libraries then use the pseudoinverse, which we name but do not develop.

The normal equation by hand

Fit $y = \theta_0 + \theta_1 x$ to the three points $(0,1), (1,2), (2,2)$. All matrices verified in `code/verify_numerics.py`.

$$\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 5 \\ 6 \end{pmatrix},$$

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{15-9} \begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix}, \quad \hat{\boldsymbol{\theta}} = \frac{1}{6} \begin{pmatrix} 25-18 \\ -15+18 \end{pmatrix} = \begin{pmatrix} 7/6 \\ 1/2 \end{pmatrix}.$$

- The fitted line $y = \frac{7}{6} + \frac{1}{2}x$ gives predictions $(1.17, 1.67, 2.17)$ with residuals $(-\frac{1}{6}, \frac{1}{3}, -\frac{1}{6})$ — which sum to zero, as least squares with an intercept guarantees.

Closed form and gradient descent agree

Both routes were run on one synthetic problem ($y = 4 + 3x + \varepsilon$, $m = 100$); frozen output of `code/05_linreg.py`:

```
normal equation:        theta = [3.9487 3.0383]
batch GD (2000 steps): theta = [3.9487 3.0383]  (identical)
```

- The estimates agree to four decimals — one convex bowl, one minimum, two routes to the same point. The estimates differ from the true $(4, 3)$ because $m = 100$ noisy points carry finite information (Lecture 1's variance, quantified).
- Costs differ, however: solving the normal equation costs on the order of d^3 operations, prohibitive for large d, while a GD step costs on the order of md. For the high-dimensional, non-quadratic objectives of deep learning, only the gradient route remains viable — which is why it was developed first.

Continuity check: the housing pipeline

Lecture 1's end-to-end project measured linear regression through the full preprocessing pipeline, by five-fold cross-validated (CV) root-mean-squared error. Re-running that measurement today, from this lecture's own code:

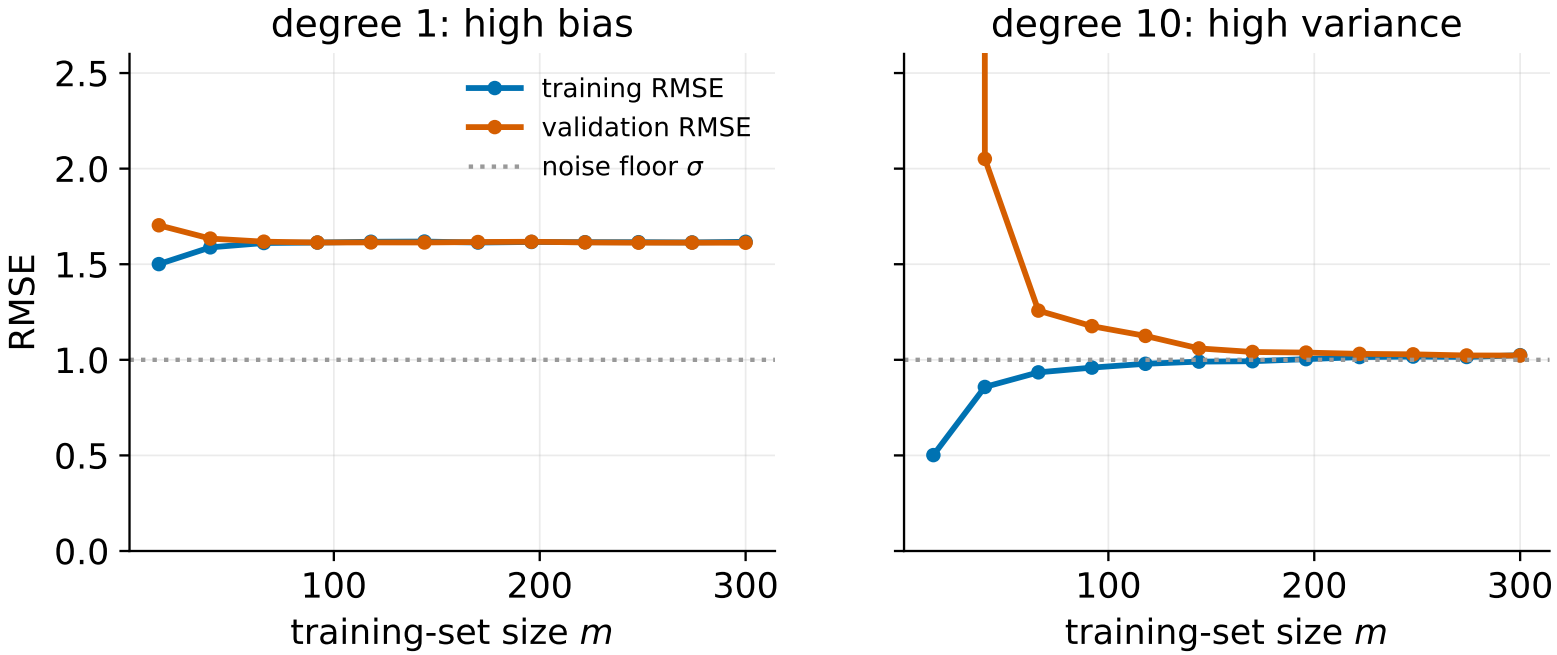
```
housing, linear regression, 5-fold CV RMSE = 68322 +- 354
(L1 frozen value: 68322 +- 354)
```

- The number reproduces exactly — same data, same split seed, same pipeline, same model. Reproducibility follows from the dataset contract and the fixed seeds.
- Today we also understand the model *inside* that pipeline: it is the normal equation applied to the imputed, scaled, one-hot encoded design matrix.

Polynomial features: capacity without new machinery

- A linear model can fit curves — by being linear in *parameters* rather than in the raw input. Mapping $x \mapsto (x, x^2, \ldots, x^k)$ and fitting linearly in the mapped features fits a degree-k polynomial in x.
- Everything from this act transfers verbatim: the design matrix gains columns, the normal equation and the gradient are unchanged as formulas.
- The degree k is precisely Lecture 1's *capacity dial*: the validation curve and the bias–variance ledger apply without modification. We now run the diagnosis on a known truth.

Learning curves for two capacities



Truth: a quadratic with noise $\sigma = 1$; models of degree 1 and 10. Generated by `code/05_linreg.py`.

- The degree-1 model plateaus *above* the noise floor with a small gap: high bias, and more data will not help — Lecture 1's diagnosis chart, first application on our own model.
- The degree-10 model shows the training curve *below* the floor (it fits noise) and a validation gap that closes slowly: high variance, reduced by data or by the next act's penalty.

Penalizing weights: the ledger's first mechanism

- Lecture 1 stated that every anti-overfitting tool is a variance-reduction device with a bias cost. The first concrete mechanism makes large weights *expensive*.
- **Ridge regression** adds a squared penalty to the objective,

$$J_{\text{ridge}}(\boldsymbol{\theta}) = \frac{1}{2m}\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \frac{\lambda}{2m}\sum_{j\geq 1}\theta_j^2,$$

  where $\lambda \geq 0$ sets the penalty strength. The bias term θ_0 is conventionally not penalized — shifting all targets should shift the intercept freely.
- At $\lambda = 0$ ridge is ordinary least squares; as $\lambda \to \infty$ all penalized weights are forced toward zero. The dial between these extremes is tuned on validation data, never on test (Lecture 1's rules).

Ridge in closed form

The penalty is quadratic, so the whole objective remains a convex quadratic. Setting the gradient to zero, with $\mathbf{A}$ the identity matrix whose top-left (bias) entry is zeroed:

$$\frac{1}{m}\mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \frac{\lambda}{m}\mathbf{A}\boldsymbol{\theta} = 0 \quad \Longleftrightarrow \quad \hat{\boldsymbol{\theta}}_{\text{ridge}} = \left(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{A}\right)^{-1}\mathbf{X}^\top\mathbf{y}.$$

- One term changed relative to the normal equation: λ is added along the diagonal. This addition also *repairs invertibility* — $\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{A}$ is invertible for $\lambda > 0$ even when features are collinear, which is the algebraic counterpart of variance reduction.
- The estimate shrinks smoothly toward zero as λ grows; no coefficient is ever set exactly to zero. The contrast with the lasso is the entire point of this pair.

Lasso: the absolute-value penalty

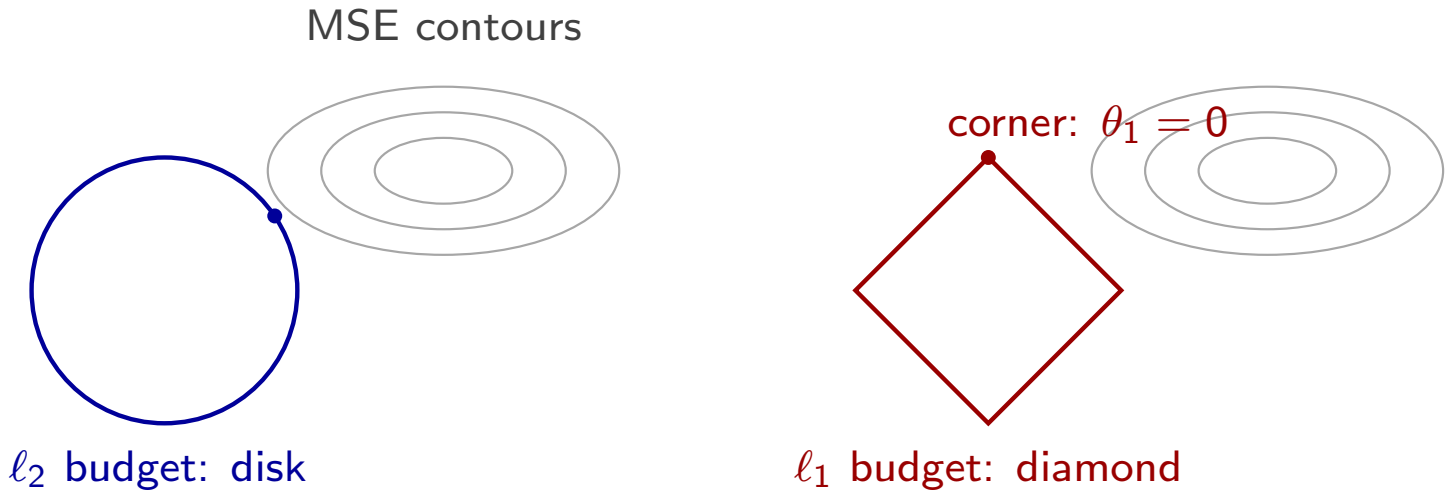
Lasso regression penalizes absolute values instead of squares:

$$J_{\text{lasso}}(\boldsymbol{\theta}) = \frac{1}{2m}\|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \frac{\lambda}{m}\sum_{j\geq 1}|\theta_j|.$$

- The penalty is not differentiable at $\theta_j = 0$, so there is no closed form and plain gradient descent needs care (subgradients, named only); scikit-learn solves it by coordinate descent.
- Against this inconvenience, lasso sets coefficients *exactly* to zero as λ grows, performing feature selection as a side effect of fitting.
- Why the kink produces exact zeros is a geometric fact; it is presented as a picture rather than an assertion.

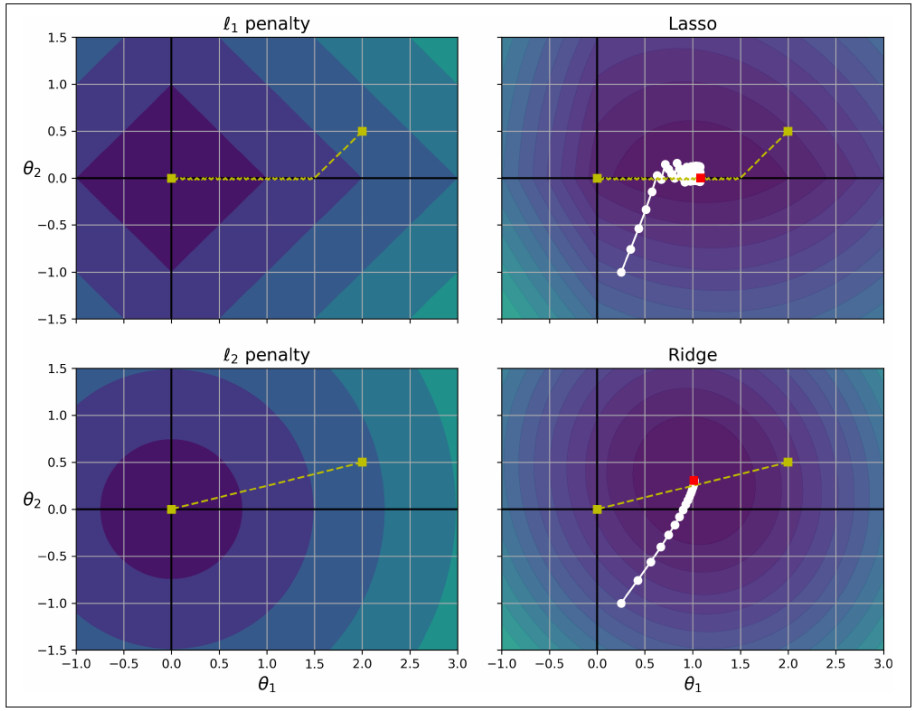
The geometric view

Penalized fitting is equivalent to *constrained* fitting — minimizing the MSE subject to a budget on the penalty (the equivalence is Lagrange duality, quoted here, not derived):



- The solution sits where the smallest MSE contour first touches the budget region. A disk is touched at a generic boundary point; a diamond is very often touched at a *corner* — and corners lie on the axes, where some coordinate is exactly zero.

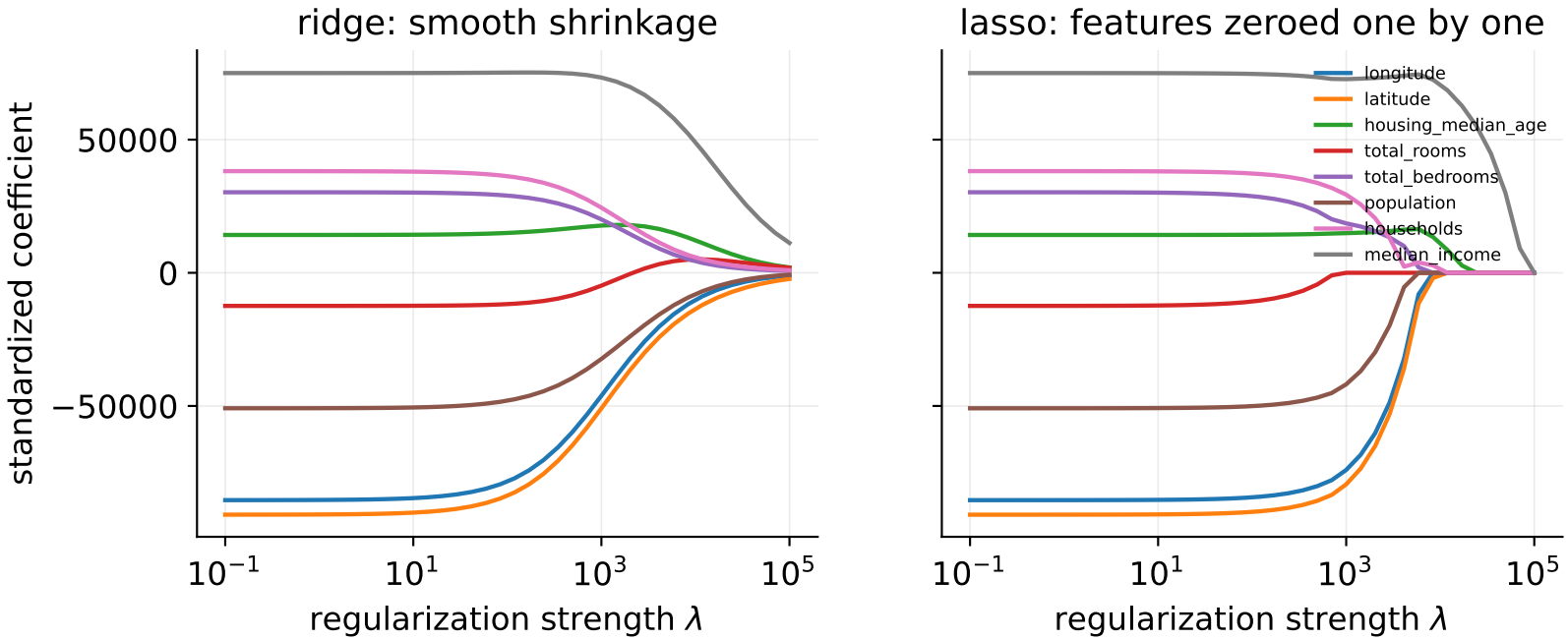
The same geometry, from the book



Géron (2025), Fig. 4-20: penalty contours (left) and regularized objectives with descent paths (right), for ℓ_1 (top) and ℓ_2 (bottom).

- In the lasso panels the descent path reaches the axis $\theta_2 = 0$ early and then travels *along* it — the corner argument, as computed by an optimizer.
- In the ridge panels the path curves smoothly toward a solution near, but never exactly on, the axes.

Shrinkage vs. sparsity on real data




```
lasso lambda=100: 8/8 nonzero    lasso lambda=10000: 4/8 nonzero
lasso lambda=1000: 7/8 nonzero   ridge lambda=10000: 8/8 nonzero
```

- On the housing numerics (standardized, training part), ridge shrinks all eight coefficients smoothly, while lasso switches them off one by one as λ grows — at $\lambda = 10^4$ half the features are gone.
- Elastic net, named for completeness, mixes the two penalties.

The two penalties, and their later reappearance

- Both penalties trade a little bias for variance reduction — an entry on the ledger, with λ as the dial tuned on validation data.
- Ridge shrinks every feature but keeps all of them; lasso deletes features outright, which aids interpretation but behaves erratically among correlated features (it picks one of a correlated group arbitrarily — a caveat to record).
- Forward reference: in deep learning the ridge penalty reappears under the name **weight decay**, attached to every layer of a network. Lecture 6 develops it — including a modern subtlety about adaptive optimizers where the equivalence breaks.

Assembling the one-neuron classifier

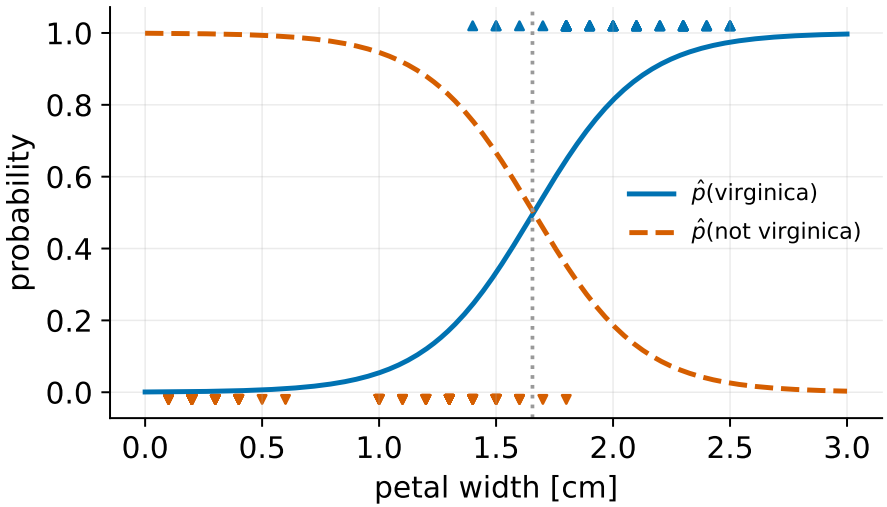
Every component has already been derived; logistic regression is their assembly:

$$\underbrace{z = \mathbf{w}^\top \mathbf{x} + b}_{\text{linear score (Act I)}} \longrightarrow \underbrace{\hat{p} = \sigma(z)}_{\text{sigmoid (Act IV)}} \longrightarrow \underbrace{J = \mathrm{BCE}(\hat{p}, y)}_{\text{Bernoulli MLE (Act IV)}}$$

trained by gradient descent (Act III) under the training-loop skeleton (Act II).

- This is, structurally, **the first neural network of the course**: one neuron — a linear map followed by a nonlinearity — trained by gradient descent on an MLE loss.
- Unlike the perceptron, it optimizes a smooth loss: on non-separable data it settles at the minimum-BCE boundary instead of cycling forever, and it outputs calibratable probabilities rather than bare signs (Lecture 1's calibration act applies).

The decision boundary is still linear



Predicting "Iris virginica" from petal width alone. The predicted probability crosses $\frac{1}{2}$ where $z = 0$, because $\sigma(0) = \frac{1}{2}$:

$$\hat{p} = \tfrac{1}{2} \iff \mathbf{w}^\top \mathbf{x} + b = 0.$$

The boundary is the same hyperplane geometry as the perceptron's — only the *training* changed. Frozen output:

```
decision boundary at
petal width = 1.66 cm
```

The sigmoid adds reported uncertainty, not flexibility: predictions near the boundary are assigned probabilities close to $\hat{p} \approx 0.5$.

The gradient of BCE through the sigmoid

We compute how the BCE loss changes with the score z, by the chain rule through $\hat{p} = \sigma(z)$, for one example:

$$\frac{\partial \mathcal{L}}{\partial \hat{p}} = \frac{\partial}{\partial \hat{p}}\Big[- y \ln \hat{p} - (1-y) \ln(1-\hat{p})\Big] = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}},$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{p}} \cdot \underbrace{\hat{p}(1-\hat{p})}_{\sigma'(z)} = -y(1-\hat{p}) + (1-y)\,\hat{p} = \hat{p} - y.$$

Foundational manipulation (reused in Lecture 3)

For sigmoid + BCE: $\partial \mathcal{L}/\partial z = \hat{p} - y$.

- The logarithm's steepness exactly cancels the sigmoid's flatness, leaving the raw *prediction error*. Adaline's update reappears — now derived from likelihood rather than assumed.

The same cancellation for softmax

For K classes with $\hat{p} = \mathrm{softmax}(\mathbf{z})$ and indicator targets $\mathbf{y}$, the per-example loss is $\mathcal{L} = -\ln \hat{p}_c = -z_c + \ln \sum_j e^{z_j}$ (using the definition of softmax). Differentiating with respect to score z_k:

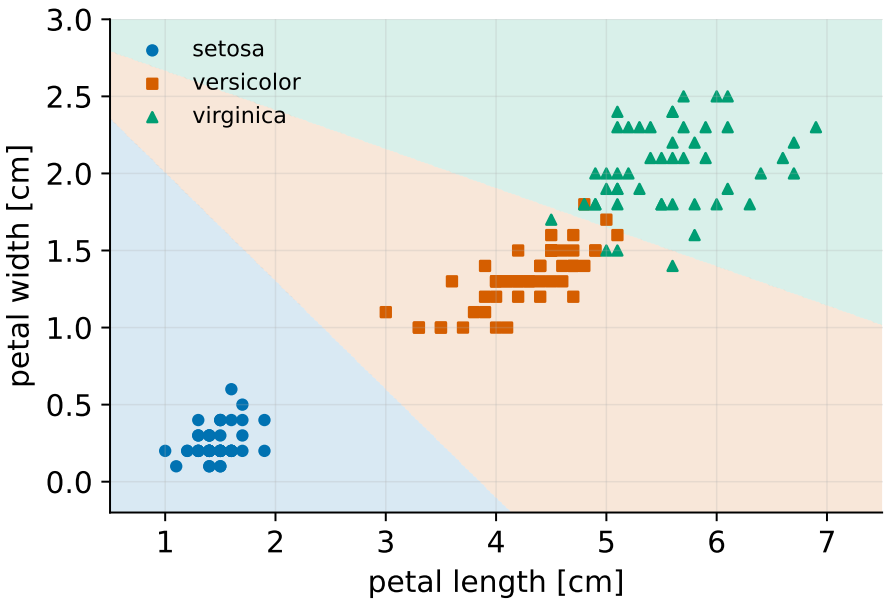
$$\frac{\partial \mathcal{L}}{\partial z_k} = -\mathbf{1}[k = c] + \frac{e^{z_k}}{\sum_j e^{z_j}} = \hat{p}_k - y_k.$$

Foundational manipulation, general form

For softmax + cross-entropy: $\partial \mathcal{L} / \partial \mathbf{z} = \hat{\mathbf{p}} - \mathbf{y}$.

- The result is prediction minus target again, for any number of classes. When Lecture 3 derives backpropagation, this is the quantity that enters the output layer, and we will quote this box verbatim.

Softmax regression on all three Iris classes



Softmax regression gives each class its own score, $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, and trains all of them jointly under the cross-entropy. Frozen output (`code/07_logistic_softmax.py`):

```
softmax regression on Iris
(3 classes, 2 features):
test accuracy = 0.911
```

The data was split and the metric declared before training — Lecture 1's rules, in force.

The class regions are separated by straight lines: each pairwise boundary is $z_k = z_l$, a linear equation. One layer of scores cannot produce a curved boundary.

Deep learning as stacked linear–nonlinear units

- Logistic regression is a *zero-hidden-layer* network: the input passes through one linear map and one nonlinearity, which produce the prediction. Softmax regression widens the output; the depth is unchanged.
- Every boundary drawn today was a straight line, and the XOR theorem proved this is not an accident of training but a ceiling of the family.
- The natural next step is to feed the outputs of neurons *into other neurons*. Lecture 3 shows that a network of three units solves XOR outright, and that the chain rule — used today through one sigmoid — scales into backpropagation, the algorithm that trains arbitrary such stacks.
- The pattern "linear map, then nonlinearity" is the course's recurring construction: it reappears as convolution (Lecture 7) and attention (Lecture 9).

Headroom: the perceptron convergence bound, sketched

Labelled a sketch; the full proof is classical. Absorb b into $\mathbf{w}$, take $\eta = 1$, and assume separability with margin γ: some unit vector $\mathbf{w}^*$ has $y\,\mathbf{w}^{*\top}\mathbf{x} \geq \gamma$ for all examples, and let $R = \max_i \|\mathbf{x}^{(i)}\|$.

- *Alignment grows linearly.* Each mistake on $(\mathbf{x}, y)$ updates $\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$, so $\mathbf{w}^{*\top}\mathbf{w}$ grows by $y\,\mathbf{w}^{*\top}\mathbf{x} \geq \gamma$. After M mistakes, $\mathbf{w}^{*\top}\mathbf{w} \geq M\gamma$.
- *Length grows slowly.* On a mistake, $y\,\mathbf{w}^{\top}\mathbf{x} \leq 0$, so $\|\mathbf{w} + y\mathbf{x}\|^2 = \|\mathbf{w}\|^2 + 2y\,\mathbf{w}^{\top}\mathbf{x} + \|\mathbf{x}\|^2 \leq \|\mathbf{w}\|^2 + R^2$. After M mistakes, $\|\mathbf{w}\|^2 \leq MR^2$.
- *Squeeze.* By Cauchy–Schwarz, $M\gamma \leq \mathbf{w}^{*\top}\mathbf{w} \leq \|\mathbf{w}\| \leq \sqrt{M}R$, hence $M \leq (R/\gamma)^2$. $\square$ (sketch)

Summary — Lecture 2: the machine

- The **perceptron** learns a separating hyperplane by error-driven updates, provably in finitely many mistakes when one exists — and provably never computes XOR, because no single linear threshold does.
- **Adaline's decisive change** — learning on the continuous score and thresholding only to predict — made the loss differentiable and created the **training-loop skeleton**: forward, loss, backward, step. Every model through Lecture 13 runs it.
- **Gradient descent** follows $-\nabla J$ because the gradient is provably the steepest direction; its single hyperparameter η produces crawl, convergence, or divergence, and we computed all three by hand.
- Feature scaling turned a divergent training run into a smooth one — Lecture 1's claim, confirmed by experiment.

Summary — Lecture 2: the losses and the models

- **Losses are theorems**: Gaussian noise yields MSE, Laplace yields MAE, Bernoulli yields binary cross-entropy, and categorical labels yield softmax cross-entropy — each as a negative log-likelihood.
- The **softmax** was defined, shift invariance derived, and numerical stability demonstrated; its cross-entropy gradient is $\hat{\mathbf{p}} - \mathbf{y}$ — the quantity that enters Lecture 3's output layer.
- **Linear regression** has a closed form — the normal equation, computed by hand and reproduced by gradient descent — and **ridge/lasso** constrain it: smooth shrinkage against exact sparsity, the corner geometry explaining the difference.
- **Logistic and softmax regression** are one-layer neural networks; all their boundaries are linear, by the XOR ceiling.

Next lecture: the multilayer perceptron

- Lecture 3 stacks today's neurons into a **multilayer perceptron** and states the modern view: a parameterized compositional function space, in which training selects a point by gradient descent.
- The XOR limitation is resolved constructively — an explicit two-neuron hidden layer plus one output neuron computes XOR, and we will draw its boundary.
- The chain rule, used today through a single sigmoid, becomes **backpropagation**: the systematic computation of all gradients in a composed function, derived completely and verified on a network small enough to compute by hand.
- Today's $\hat{\mathbf{p}} - \mathbf{y}$ will be the first quantity that derivation produces at the output layer.